

Справочник по архитектурам (раздаточный материал)

Справочный материал к модулю про линейное внимание: два числа состояния для девяти архитектур, конфиги, по которым они посчитаны, и источники.

Два числа

Каждой архитектуре в этом модуле сопоставлены два числа; они определены один раз и везде используются под этими названиями и никакими другими.

Байты состояния на слой при длине контекста N. Сколько памяти должен удерживать один слой после N токенов, чтобы выдать следующий токен.

- Softmax-внимание, на слой: $2 * n_kv_heads * head_dim * dtype_bytes * N$ байт — **линейно по N**.
- Каждая архитектура линейного внимания, на слой: $N * d_k * d_v * dtype_bytes$ байт — **не зависит от N**.
- Гибриды — сумма по их слоям.

Прирост байтов на токен, на слой. На сколько это число растёт, когда приходит ещё один токен.

- Softmax-внимание: $2 * n_kv_heads * head_dim * dtype_bytes$ байт на токен — **константа, и она ненулевая**.
- Каждая архитектура линейного внимания: **0 байт на токен**.

Формулировка «байты состояния на токен» запрещена. Для softmax-внимания это число-прирост, а для всего остального — бессмысленная величина вида $1/N$, и одна подпись на две разные единицы — ровно то, из-за чего контраст ниже перестаёт сходиться. Каждый раз говорите, которое из двух чисел вы имеете в виду.

Ненулевой прирост против нулевого — вот предмет этого модуля.

Разобранные эталонные числа

Эталонная модель лекции, пересчитанная заново: **по образцу Llama-3-8B: 32 слоя, $n_kv_heads = 8$, $head_dim = 128$, fp16**, против линейного слоя той же ширины ($N = 32$, $d_k = d_v = 128$, fp16).

- Softmax — прирост байтов на токен, на слой: **4 KiB (4,096 B)**
- Softmax — байты состояния на слой при $N = 128k$: **512 MiB (536,870,912 B)**
- Softmax — состояние по всей модели при $N = 128k$ (32 слоя): **16 GiB (17,179,869,184 B)**
- Линейный слой — байты состояния на слой, при любом N: **1 MiB (1,048,576 B)**
- Линейный слой — состояние по всей модели (32 слоя), при любом N: **32 MiB (33,554,432 B)**
- Линейный слой — прирост байтов на токен, на слой: **0 B**
- Разрыв на слой при $N = 128k$: **512x**
- Ловушка n_heads (32 KV-головы вместо 8): **64 GiB (68,719,476,736 B)**

Разрыв — 512x на слой при 128k, и он не ограничен по N: число линейного слоя не двигается с места.

Таблица

Длины контекста: $4k = 4,096$, $32k = 32,768$, $128k = 131,072$. Каждая строка посчитана при $dtype_bytes = 2$; конфиги и типы данных каждой строки — в следующей таблице.

Архитектура	Форма затухания	Правило обновления	Нормализация	Байты состояния на слой при N	Прирост байтов на токен, на слой
Softmax-внимание (эталон)	нет — кэш точный, ничего не забывается	дописать (k_t, v_t) в кэш; $o_t = \text{softmax}(q_t K^T / \sqrt{d_k}) V$	softmax по оценкам внимания, с масштабированием на $1/\sqrt{d_k}$	16 MiB при 4k / 128 MiB при 32k / 512 MiB при 128k (GQA self-attention, линейно по N)	4 KiB (4,096 B) (GQA self-attention)

Архитектура	Форма затухания	Правило обновления	Нормализация	Байты состояния на слой при N	Прирост байтов на токен, на слой
Linear Transformer¹	нет ($\alpha = 1$); состояние только растёт	аддитивное: $S_t = S_{t-1} + \text{phi}(k_t) v_t^T$	знаменатель $\text{sum-phi}(k)$ (накопительный вектор z_t), отображение признаков phi	2 MiB при любом N (линейное внимание)	0 В (линейное внимание)
RetNet	фиксированный скаляр на голову, γ_h — не зависит от данных, не выучивается на каждый токен	аддитивное с гейтом: $S_t = \gamma_h S_{t-1} + k_t v_t^T$	GroupNorm/RMSNorm на выходе по головам, плюс выходной гейт swish ; знаменателя $\text{sum-phi}(k)$ нет	2 MiB при любом N (retention)	0 В (retention)
GLA	поканальный вектор, зависящий от входа: $\text{Diag}(\alpha_t)$, по гейту на каждый канал ключа	аддитивное с гейтом: $S_t = \text{Diag}(\alpha_t) S_{t-1} + k_t v_t^T$	RMSNorm на выходе плюс выходной гейт ($\text{use_output_gate} = \text{true}$); знаменателя нет	1 MiB при любом N (линейное внимание с гейтом)	0 В (линейное внимание с гейтом)
Mamba2	зависящий от входа скаляр на голову: $\alpha_t = \exp(-\Delta_t * a_h)$	аддитивное с гейтом (SSD): $S_t = \alpha_t S_{t-1} + k_t v_t^T$, где $k_t = B_t, v_t = x_t$	RMSNorm с гейтом на выходе ($\text{rmsnorm} = \text{True}$), плюс короткая depthwise-свёртка на входе; знаменателя нет	1.25 MiB при любом N (Mamba2 SSD)	0 В (Mamba2 SSD)
DeltaNet	нет — члена затухания нет вовсе	дельта-правило: $S_t = (I - \beta_t k_t k_t^T) S_{t-1} + \beta_t k_t v_t^T$ (стирающий множитель слева)	L2-нормированные q и k ($qk_{\text{norm}} = 'l2'$) после SiLU, короткая depthwise-свёртка, RMSNorm на выходе; выходного гейта нет ($\text{use_gate} = \text{false}$)	512 KiB при любом N (дельта-правило)	0 В (дельта-правило)
Gated DeltaNet	зависящий от входа скаляр на голову, α_t (в стиле Mamba2, из параметризации через $\text{softplus}/\exp$)	дельта-правило с гейтом: $S_t = \alpha_t (I - \beta_t k_t k_t^T) S_{t-1} + \beta_t k_t v_t^T$	L2-нормированные q/k после SiLU, короткая depthwise-свёртка на $q/k/v$, RMSNorm с гейтом на выходе	1.37 MiB при любом N (дельта-правило с гейтом)	0 В (дельта-правило с гейтом)

¹**Linear Transformer** — конфиг не привязан к выпущенной модели. Оригинальная статья не публикует конфигурации масштаба языковой модели. Её самая большая модель — 16-слойный 8-головый авторегрессионный трансформер для изображений CIFAR-10 с размером эмбединга 256, то есть $d_k = d_v = 32$ на голову (arXiv 2006.16236, раздел 4.2.2, который переиспользует конфигурацию MNIST из раздела 4.2.1), и dtype в ней не указан. Чекпоинта `linear_attn` на `flax-hub` нет — на практике семейство вытеснено своими потомками с гейтами. Поэтому числа в этой строке вычислены по значениям по умолчанию класса `LinearAttentionConfig` из `flash-linear-attention` при форме 1.3B, общей для всех остальных строк FLA, и читать их нужно как «сколько эта архитектура стоила бы при такой ширине», а не как измерение обученной модели.

Архитектура	Форма затухания	Правило обновления	Нормализация	Байты состояния на слой при n	Прирост байтов на токен, на слой
Gated DeltaNet-2²	поканальный вектор $\alpha_t = \exp(g_t)$, $g_t = -\exp(a) * \text{softplus}(W_f x_t + \text{delta})$	дельта-правило-2 с гейтом (Gated Delta Rule-2), стирание и запись разделены: $S_t = (I - k_t (b_t \cdot k_t)^T) \text{Diag}(\alpha_t) S_{t-1} + k_t (w_t \cdot v_t)^T$, с поканальным гейтом стирания b_t in $[0,1]^{d_k}$ и поканальным гейтом записи w_t in $[0,1]^{d_v}$	L2-нормированные q/k после короткой свёртки + SiLU, RMSNorm на выходе с выходным гейтом на SiLU	512 KiB при любом n (дельта-правило-2 с гейтом)	0 В (дельта-правило-2 с гейтом)
KDA (Kimi Linear)³⁴	поканальное (тонкозернистое) диагональное затухание $\text{Diag}(\alpha_t)$, переход DPLR с ограничениями; у слоёв полного внимания затухания нет	дельта-правило с гейтом и поканальным затуханием (KDA) на 20 слоях; softmax-MLA на 7 слоях	L2-нормированные q/k, короткая свёртка (ядро 4), RMSNorm на выходе с сигмоидным выходным гейтом; NoPE на всех слоях MLA	1 MiB при любом n (KDA (линейное))+ 4.50 MiB при 4k / 36 MiB при 32k / 144 MiB при 128k (полное внимание MLA, линейно по n)	0 В (KDA (линейное))+ 1,152 В (полное внимание MLA)
Qwen3-Next⁵	зависящий от входа скаляр на голову значения $g_t = -\exp(A_{\log}) * \text{softplus}(a_t + dt_{\text{bias}})$; у слоёв полного внимания затухания нет	дельта-правило с гейтом $\text{Qwen3NextGatedDeltaNet}$ на 36 слоях; softmax-внимание с GQA на 12 слоях	L2-нормированные q/k внутри ядра, короткая depthwise-свёртка (ядро 4), RMSNorm с гейтом на выходе; слои внимания используют частичный RoPE (25%) и выход с гейтом	1 MiB при любом n (Gated DeltaNet (линейное))+ 8 MiB при 4k / 64 MiB при 32k / 256 MiB при 128k (полное внимание GQA, линейно по n)	0 В (Gated DeltaNet (линейное))+ 2 KiB (2,048 В) (полное внимание GQA)

²**Gated DeltaNet-2 — тип данных состояния.** Приложение D.3: 'The recurrent state is stored in fp32 across chunks and during recurrent decoding.' При fp32 состояние этой строки на слой — 1.00 MiB, а не 512 KiB. В таблице эта строка, как и все остальные, посчитана по 2 байта, чтобы сравнивалась ширина; в реальном развёртывании увидят именно цифру для fp32.

³Гибридная строка — см. раздел *Две гибридные строки, разложенные явно выше*.

⁴**KDA (Kimi Linear) — член, линейный по N, здесь MLA, а не GQA.** Член, линейный по N, здесь — MLA, который HE подчиняется GQA-формуле из раздела «Два числа». MLA кэширует один сжатый латент на токен на слой, шириной $kv_lora_rank + qk_rope_head_dim = 512 + 64 = 576$, общий для всех 32 голов: 1152 байта на токен на слой MLA. Именно на таком учёте держится заявление о 75%. Эталонная HF-реализация (modeling_kimi.py, KimiMLAAttention.forward) вместо этого материализует в кэш развёрнутые K и V по головам — $32 * (192 + 128) = 10,240$ элементов, 20,480 байт на токен на слой, ~18x больше. Эффективный 'absorbed'-путь, который используется в vLLM, кэширует латент. Цифра 75% определяется соотношением слоёв и одинакова при любом из двух учётов.

⁵Гибридная строка — см. раздел *Две гибридные строки, разложенные явно выше*.

Откуда взяты числа каждой строки

Каждая строка называет конфиг, по которому она вычислена, и свой dtype_bytes. Столбцы «по всей модели» — это числа на слой, просуммированные по слоям модели: то, что блок 0 лекции считает вживую и что нужно рассуждению о гибридах в блоке 6.

Архитектура	Конфиг, по которому вычислено	dtype_bytes	Слои	Формула на слой	Состояние по всей модели при 4k / 32k / 128k	Прирост по всей модели
Softmax-внимание (эталон)	Llama-3-8B (meta-llama/ Meta-Llama-3-8B) (опубликованный чекпоинт)	2	$32 \times \text{GQA} \times \text{self-attention}$	$\text{heads} \times \text{head_dim} \times \text{dtype_bytes} = 2 \times 8 \times 128 \times 2 \times N$	512 MiB / 4 GiB / 16 GiB	128 KiB
Linear Transformer	flash-linear-attention LinearAttentionConfig, значения по умолчанию (форма 1.3B) (эталонный конфиг из библиотеки — HE выпущенный чекпоинт)	2	$24 \times \text{линейное внимание}$	$H \times d_k \times d_v \times \text{dtype_bytes} = 4 \times 512 \times 512 \times 2$	48 MiB / 48 MiB / 48 MiB	0 B
RetNet	fla-hub/ retnet-1.3B-100B (опубликованный чекпоинт)	2	$24 \times \text{retention}$	$H \times d_k \times d_v \times \text{dtype_bytes} = 8 \times 256 \times 512 \times 2$	48 MiB / 48 MiB / 48 MiB	0 B
GLA	fla-hub/ gla-1.3B-100B (опубликованный чекпоинт)	2	$24 \times \text{линейное внимание с гейтом}$	$H \times d_k \times d_v \times \text{dtype_bytes} = 4 \times 256 \times 512 \times 2$	24 MiB / 24 MiB / 24 MiB	0 B
Mamba2	state-spaces/ mamba2-2.7b (опубликованный чекпоинт)	2	$64 \times \text{Mamba2 SSD}$	$H \times d_k \times d_v \times \text{dtype_bytes} = 80 \times 128 \times 64 \times 2$	80 MiB / 80 MiB / 80 MiB	0 B
DeltaNet	fla-hub/ delta_net-1.3B-100B (опубликованный чекпоинт)	2	$24 \times \text{дельта-правило}$	$H \times d_k \times d_v \times \text{dtype_bytes} = 16 \times 128 \times 128 \times 2$	12 MiB / 12 MiB / 12 MiB	0 B
Gated DeltaNet	NVlabs/ GatedDeltaNet, конфиг GatedDeltaNet_1.3B (опубликованный конфиг обучения (официальный релиз кода; чекпоинта на HF нет))	2	$16 \times \text{дельта-правило с гейтом}$	$H \times d_k \times d_v \times \text{dtype_bytes} = 9 \times 200 \times 400 \times 2$	21.97 MiB / 21.97 MiB / 21.97 MiB	0 B
Gated DeltaNet-2	NVlabs/ GatedDeltaNet-2, конфиг gdn2_1.3B; размерности	2	$18 \times \text{дельта-правило-2 с гейтом}$	$H \times d_k \times d_v \times \text{dtype_bytes} = 16 \times 128 \times 128 \times 2$	9 MiB / 9 MiB / 9 MiB	0 B

Архитектура	Конфиг, по которому вычислено	dtype_bytes	Слои	Формула на слой	Состояние по всей модели при 4k / 32k / 128k	Прирост по всей модели
	голов сверены с приложением E.1 arXiv 2605.22791 (опубликованный конфиг обучения (официальный релиз кода; чекпоинта на HF net))					
KDA (Kimi Linear)	moonshotai/Kimi-Linear-48B-A3B-Instruct (48B всего, 3B активных, MoE) (опубликованный чекпоинт)	2	20 × KDA (линейное) + 7 × полное внимание MLA	$H*d_k*d_v*dtype_bytes = 32*128*128*2*latent_dim*dtype_bytes*N = 576*2*N$	51.50 MiB / 272 MiB / 1.00 GiB	8,064 B
Qwen3-Next	Qwen/Qwen3-Next-80B-A3B-Instruct (80B всего, 3B активных, MoE) (опубликованный чекпоинт)	2	36 × Gated DeltaNet (линейное) + 12 × полное внимание GQA	$H*d_k*d_v*dtype_bytes = 32*128*128*2*2*n_kv_heads*head_dim*dtype_bytes*N = 2*2*256*2*N$	132 MiB / 804 MiB / 3.04 GiB	24 KiB

Две гибридные строки, разложенные явно

Гибрид — это ровно та строка, которую нельзя описать одним столбцом: **постоянный член** от его линейных слоёв плюс **член, линейный по N**, от его слоёв полного внимания. Здесь оба выписаны отдельно, а не свёрнуты в сумму.

KDA (Kimi Linear) — moonshotai/Kimi-Linear-48B-A3B-Instruct (48B всего, 3B активных, MoE)

Соотношение, заявленное в статье: **3:1 (KDA : полное MLA)**. Соотношение в выпущенном чекпоинте: **20:7 = 2.86:1 на 27 слоях**.

- **Постоянный член** — 20 × KDA (линейное): $H*d_k*d_v*dtype_bytes = 32*128*128*2 = 1 \text{ MiB}$ (1,048,576 B) на слой, при любом N, **20 MiB по всей модели**. Прирост: **0 B на токен**.
- **Член, линейный по N** — 7 × полное внимание MLA: прирост **1,152 B на токен на слой**, то есть 4.50 MiB при 4k, 36 MiB при 32k, 144 MiB при 128k на слой (31.50 MiB при 4k, 252 MiB при 32k, 1008 MiB при 128k по всей модели).
- **Прирост по всей модели: 8,064 B на токен** — целиком от 7 слоёв полного внимания.
- **Заявление о сокращении KV-кэша**. Базовый вариант (для заявленных 75% сравниваем с полным MLA на всех 27 слоях): $27 * 1,152 \text{ B} = 31,104 \text{ B на токен}$. Эта модель: **8,064 B на токен**. Сокращение = **74.1%**. Это утверждение о числе-приросте и ни о чём другом — постоянный член от линейных слоёв им не затрагивается, и никакой «размер состояния на токен» здесь не заявляется.

Qwen3-Next — Qwen/Qwen3-Next-80B-A3B-Instruct (80B всего, 3B активных, MoE)

Соотношение, заявленное в статье: **3:1 (линейное : полное внимание)**. Соотношение в выпущенном чекпоинте: **36:12 = ровно 3:1**.

- **Постоянный член** — $36 \times \text{Gated DeltaNet}$ (линейное): $H*d_k*d_v*dtype_bytes = 32*128*128*2 = 1 \text{ MiB}$ (1,048,576 В) на слой, при любом n , **36 MiB по всей модели**. Прирост: **0 В на токен**.
- **Член, линейный по n** — $12 \times$ полное внимание GQA: прирост **2 KiB (2,048 В) на токен на слой**, то есть 8 MiB при 4k, 64 MiB при 32k, 256 MiB при 128k на слой (96 MiB при 4k, 768 MiB при 32k, 3 GiB при 128k по всей модели).
- **Прирост по всей модели: 24 KiB (24,576 В) на токен** — целиком от 12 слоёв полного внимания.
- **Заявление о сокращении KV-кэша**. Базовый вариант (для заявленных 75% сравниваем с полным вниманием (GQA) на всех 48 слоях): $48 * 2,048 \text{ в} = 96 \text{ KiB (98,304 В) на токен}$. Эта модель: **24 KiB (24,576 В) на токен**. Сокращение = **75.0%**. Это утверждение о числе-приросте и ни о чём другом — постоянный член от линейных слоёв им не затрагивается, и никакой «размер состояния на токен» здесь не заявляется.

Примечания и ссылки на источники

Проверка цитирований и утверждений

Каждое число, взятое из источника, один раз сверено с этим источником.

Утверждение	Вердикт	Что на самом деле говорит источник
arXiv 2507.19595 — это <i>Efficient Attention Mechanisms for LLMs: A Survey</i>	разрешается; название сокращено	Efficient Attention Mechanisms for Large Language Models: A Survey — Yutao Sun, Zhenyu Li, Yike Zhang, Tengyu Pan, Bowen Dong, Yuyi Guo, Jianyong Wang; submitted 2025-07-25
arXiv 2406.06484 — это <i>Parallelizing Linear Transformers with the Delta Rule over Sequence Length</i>	разрешается точно	Parallelizing Linear Transformers with the Delta Rule over Sequence Length — Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, Yoon Kim; submitted 2024-06-10
arXiv 2510.26692 — это <i>Kimi Linear</i>	разрешается точно	Kimi Linear: An Expressive, Efficient Attention Architecture — Kimi Team (Yu Zhang, Zongyu Lin, Xingcheng Yao, ... , Yulun Du); submitted 2025-10-30
arXiv 2605.22791 — это <i>Gated DeltaNet-2: Decoupling Erase and Write in Linear Attention</i>	подтверждено: статья существует; идентификатор, название и авторство совпадают	Gated DeltaNet-2: Decoupling Erase and Write in Linear Attention — Ali Hatamizadeh, Yejin Choi, Jan Kautz (NVIDIA); arXiv:2605.22791v1 [cs.AI], submitted 2026-05-21. Code: https://github.com/NVlabs/GatedDeltaNet-2 (HTTP 200 on 2026-09-05; it carries an official gdn2_1.3B training config and the GDN-2 layer implementation, which is where this row's layer count comes from).
Сокращение KV-кэша на 75% у Kimi Linear — утверждение о числе-приросте, а не о каком-либо размере состояния на токен	подтверждено, и арифметика сходится	The paper's wording is 'reduces memory and KV-cache usage by up to 75% during long-sequence generation' (introduction) and 'reducing KV cache usage by up to 75%' (abstract). KV-cache usage is the quantity that grows with sequence length, i.e. the marginal number. It closes exactly as a layer ratio: at a clean 3:1 only one layer in four carries an N-linear cache, giving 25% of full attention and a 75% reduction. The shipped 27-layer checkpoint is 20:7, giving 8,064 vs 31,104 bytes per token = 74.1%. Nothing in the claim concerns the constant per-

Утверждение	Вердикт	Что на самом деле говорит источник
		layer KDA state, which does not move with N at all.
Гибридное соотношение Kimi Linear — 3:1	3:1 в статье; 20:7 (2.86:1) в выпущенном чекпоинте	Paper: 'Kimi Linear interleaves KDA with periodic full attention layers in a uniform 3:1 ratio'; the section-5.2 ablation puts 3:1 at training ppl 9.23 / validation 5.65, against 0:1, 1:1, 7:1 and 15:1. Checkpoint: moonshotai/Kimi-Linear-48B-A3B-Instruct has num_hidden_layers = 27 with full_attn_layers = [4, 8, 12, 16, 20, 24, 27] — the 3:1 pattern over the first 24 layers, plus a seventh full-attention layer at the top. Teach 3:1; the handout prints 20:7 because that is what ships.
Гибридное соотношение Qwen3-Next — 3:1, а его линейные слои — Gated DeltaNet при H=32, d_k=d_v=128	подтверждено по исходникам HF и опубликованному конфигу	transformers configuration_qwen3_next.py builds layer_types as ['linear_attention' if bool((i + 1) % full_attention_interval) else 'full_attention'], and the published config sets full_attention_interval = 4 with num_hidden_layers = 48: 12 full-attention layers, 36 linear, exactly 3:1. linear_num_key_heads = 16 but linear_num_value_heads = 32, and modular_qwen3_next.py repeat_interleave's q and k by 2 before the recurrence, so the state carries 32 heads of 128 x 128 — not 16. That is the reference linear layer of the lecture, to the byte.
Числа лекции для Llama-3-8B и эталонного линейного слоя: прирост 4 KiB/токен/слой, 512 MiB на слой при 128k, 16 GiB по всей модели, 1 MiB и 32 MiB для линейного слоя, разрыв 512x на слой при 128k	все восемь пересчитаны, и все восемь сходятся	Пересчитаны в разделе <i>Разобранные эталонные числа</i> выше из тех же констант, что и таблица.

Подробности по строкам

Softmax-внимание (эталон)

- **Конфиг:** Llama-3-8B (meta-llama/Meta-Llama-3-8B) (опубликованный чекпоинт).
- **dtype_bytes** = 2 — config.json torch_dtype = bfloat16; в лекции — fp16. Те же 2 байта в любом случае.
- **Статья:** arXiv 2407.21783 — The Llama 3 Herd of Models
- **Источник конфига:** <https://huggingface.co/meta-llama/Meta-Llama-3-8B/blob/main/config.json>
- **Зеркало конфига** (канонический репозиторий требует авторизации): <https://huggingface.co/NousResearch/Meta-Llama-3-8B/raw/main/config.json>
- **Примечание:** GQA, а не MHA: num_attention_heads = 32, но num_key_value_heads = 8. Кэшируются только KV-головы. Если подставить сюда n_heads, получится 64 GiB по всей модели при 128k вместо правильных 16 GiB — та самая ошибка, от которой предостерегает раздел 0 лекции.
- **Ссылки на источники:**
 - num_hidden_layers = 32, num_attention_heads = 32, num_key_value_heads = 8, hidden_size = 4096, torch_dtype = bfloat16 — Llama-3-8B config.json. Канонический репозиторий meta-llama/Meta-Llama-3-8B требует авторизации (HTTP 401 без учётных данных), поэтому значения считаны 2026-09-05 с открытого зеркала NousResearch/Meta-Llama-3-8B; это широко опубликованные числа Llama-3-8B, и они совпадают с разделом 0 лекции.

- ▶ $\text{head_dim} = \text{hidden_size} / \text{num_attention_heads} = 4096 / 32 = 128$ (в этой версии у LlamaConfig нет поля head_dim; transformers выводит его сам)

Linear Transformer

- **Конфиг:** flash-linear-attention LinearAttentionConfig, значения по умолчанию (форма 1.3B) (эталонный конфиг из библиотеки — НЕ выпущенный чекпоинт).
- **dtype_bytes = 2** — не опубликован: класс конфига не несёт dtype; принято 2 байта, как у каждого чекпоинта fla-hub на 1.3B (torch_dtype = bfloat16).
- **Статья:** arXiv 2006.16236 — Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention (Katharopoulos, Vyas, Pappas, Fleuret, ICML 2020)
- **Источник конфига:** https://github.com/fla-org/flash-linear-attention/blob/64efbae863fdaf76fa5359e517b31cf1d72525a1/fla/models/linear_attn/configuration_linear_attn.py
- **Примечание:** feature_map = 'elementwise_product' (HadamardFeatureMap) сохраняет размерность головы, так что размерность признаков равна d_k и состояние остаётся d_k x d_v.
- **НЕ ПРИВЯЗАНО К ВЫПУЩЕННОЙ МОДЕЛИ:** Оригинальная статья не публикует конфигурации масштаба языковой модели. Её самая большая модель — 16-слойный 8-головый авторегрессионный трансформер для изображений CIFAR-10 с размером эмбединга 256, то есть d_k = d_v = 32 на голову (arXiv 2006.16236, раздел 4.2.2, который переиспользует конфигурацию MNIST из раздела 4.2.1), и dtype в ней не указан. Чекпоинта linear_attn на fla-hub нет — на практике семейство вытеснено своими потомками с гейтами. Поэтому числа в этой строке вычислены по значениям по умолчанию класса LinearAttentionConfig из flash-linear-attention при форме 1.3B, общей для всех остальных строк FLA, и читать их нужно как «сколько эта архитектура стоила бы при такой ширине», а не как измерение обученной модели.
- **Не учтено в столбце состояния:** нормализатор sum-phi(k), вектор z_t (H * d_k элементов) — ещё 4 KiB (4,096 B) на каждый слой типа «линейное внимание», и это не зависит от M. Формула из раздела «Два числа» этого не включает.
- **Ссылки на источники:**
 - ▶ hidden_size = 2048, num_hidden_layers = 24, num_heads = 4, expand_k = 1.0, expand_v = 1.0, feature_map = 'elementwise_product' — fla/models/linear_attn/configuration_linear_attn.py, значения по умолчанию в LinearAttentionConfig.init
 - ▶ d_k = hidden_size * expand_k / num_heads = 512, d_v = hidden_size * expand_v / num_heads = 512 — fla/layers/linear_attn.py, строки 56-68
 - ▶ самый большой конфиг оригинальной статьи: 16 слоёв, 8 голов, эмбединг 256 (32 измерения на голову) — arXiv 2006.16236, разделы 4.2.1-4.2.2

RetNet

- **Конфиг:** fla-hub/retnet-1.3B-100B (опубликованный чекпоинт).
- **dtype_bytes = 2** — config.json torch_dtype = bfloat16.
- **Статья:** arXiv 2307.08621 — Retentive Network: A Successor to Transformer for Large Language Models
- **Источник конфига:** <https://huggingface.co/fla-hub/retnet-1.3B-100B/raw/main/config.json>
- **Примечание:** Голова значений вдвое шире головы ключей: expand_v = 2 против expand_k = 1 — это выбор самой статьи и причина, по которой состояние в этой строке 2 MiB, а не 1 MiB.
- **Ссылки на источники:**
 - ▶ hidden_size = 2048, num_hidden_layers = 24, num_heads = 8, expand_k = 1, expand_v = 2, use_output_gate = true, torch_dtype = bfloat16 — fla-hub/retnet-1.3B-100B config.json
 - ▶ d_k = hidden_size * expand_k / num_heads = 256, d_v = hidden_size * expand_v / num_heads = 512 — fla/layers/multiscale_retention.py, строки 108-121

GLA

- **Конфиг:** fla-hub/gla-1.3B-100B (опубликованный чекпоинт).
- **dtype_bytes = 2** — config.json torch_dtype = bfloat16.
- **Статья:** arXiv 2312.06635 — Gated Linear Attention Transformers with Hardware-Efficient Training
- **Источник конфига:** <https://huggingface.co/fla-hub/gla-1.3B-100B/raw/main/config.json>

- **Примечание:** `use_gk = true`, `use_gv = false`: затухание только по оси ключей. `expand_k = 0.5` вдвое сужает ключи относительно ширины модели — именно поэтому состояние GLA вдвое меньше, чем у RetNet при том же hidden size.
- **Ссылки на источники:**
 - ▶ `hidden_size = 2048`, `num_hidden_layers = 24`, `num_heads = 4`, `expand_k = 0.5`, `expand_v = 1`, `use_gk = true`, `use_gv = false`, `use_output_gate = true`, `torch_dtype = bfloat16` — [fla-hub/gla-1.3B-100B config.json](https://huggingface.co/fla-hub/gla-1.3B-100B/blob/main/config.json)
 - ▶ $d_k = 2048 * 0.5 / 4 = 256$, $d_v = 2048 * 1 / 4 = 512$

Mamba2

- **Конфиг:** `state-spaces/mamba2-2.7b` (опубликованный чекпоинт).
- `dtype_bytes = 2` — не опубликован: в `config.json` чекпоинта нет поля `dtype`; для сопоставимости принято 2 байта.
- **Статья:** [arXiv 2405.21060](https://arxiv.org/abs/2405.21060) — Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality
- **Источник конфига:** <https://huggingface.co/state-spaces/mamba2-2.7b/raw/main/config.json>
- **Примечание:** Соответствие обозначениям модуля: `H = nheads`, $d_k = d_{state}$, $d_v = headdim$. Состояние имеет форму $(nheads, headdim, d_{state})$, а `B_t` из SSM играет роль ключа. Обратите внимание на асимметрию: Mamba2 получает число голов за счёт маленького `d_v` (64), тогда как семейство дельта-правил выбирает ширину, полагая $d_k = d_v = 128$.
- **Не учтено в столбце состояния:** состояние короткой свёртки, $(d_{inner} + 2n_{groups}d_{state}) * d_{conv}$ элементов — ещё 42 KiB (43,008 B) на каждый слой типа «Mamba2 SSD», и это не зависит от `n`. Формула из раздела «Два числа» этого не включает.
- **Ссылки на источники:**
 - ▶ `d_model = 2560`, `n_layer = 64`, `ssm_cfg = {layer: Mamba2}` — `state-spaces/mamba2-2.7b config.json`
 - ▶ `d_state = 128`, `d_conv = 4`, `expand = 2`, `headdim = 64`, `ngroups = 1` — значения по умолчанию Mamba2.`init`, `mamba_ssm/modules/mamba2.py` (`config.json` ни одно из них не переопределяет)
 - ▶ $d_{inner} = expand * d_{model} = 5120$; `nheads = d_{inner} / headdim = 80` — тот же источник

DeltaNet

- **Конфиг:** `fla-hub/delta_net-1.3B-100B` (опубликованный чекпоинт).
- `dtype_bytes = 2` — `config.json torch_dtype = bfloat16`.
- **Статья:** [arXiv 2406.06484](https://arxiv.org/abs/2406.06484) — Parallelizing Linear Transformers with the Delta Rule over Sequence Length
- **Источник конфига:** https://huggingface.co/fla-hub/delta_net-1.3B-100B/raw/main/config.json
- **Примечание:** Самое узкое состояние в таблице при этой ширине: 16 голов по 128x128. `beta_t` — сигмоида (`use_beta = true`), строго меньше 1.
- **Не учтено в столбце состояния:** состояние короткой свёртки на $q/k/v$, $(2 * key_dim + value_dim) * conv_size$ элементов — ещё 48 KiB (49,152 B) на каждый слой типа «дельта-правило», и это не зависит от `n`. Формула из раздела «Два числа» этого не включает.
- **Ссылки на источники:**
 - ▶ `hidden_size = 2048`, `num_hidden_layers = 24`, `num_heads = 16`, `expand_k = 1`, `expand_v = 1`, `qk_norm = 'l2'`, `qk_activation = 'silu'`, `use_short_conv = true`, `conv_size = 4`, `use_beta = true`, `use_gate = false`, `use_output_norm = true`, `torch_dtype = bfloat16` — `fla-hub/delta_net-1.3B-100B config.json`
 - ▶ $d_k = 2048 / 16 = 128$, $d_v = 2048 / 16 = 128$

Gated DeltaNet

- **Конфиг:** `NVlabs/GatedDeltaNet`, конфиг `GatedDeltaNet_1.3B` (опубликованный конфиг обучения (официальный релиз кода; чекпоинта на HF нет)).
- `dtype_bytes = 2` — `pretrain.py: L.Fabric(..., precision='bf16-mixed')`.
- **Статья:** [arXiv 2412.06464](https://arxiv.org/abs/2412.06464) — Gated Delta Networks: Improving Mamba2 with Delta Rule
- **Источник конфига:** https://github.com/NVlabs/GatedDeltaNet/blob/b53d6d3a161267432a79c1c04af69fa52bddc921/lit_gpt/config.py
- **Примечание:** Ловушка, о которой стоит знать: `n_head = 16` в конфиге — НЕ число голов миксера. `lit_gpt/model.py` строит миксер как `GatedDeltaNet(hidden_size=config.n_embd)` и ничего больше, так что

действуют собственные значения по умолчанию слоя — $\text{num_heads} = 9$, $\text{expand_k} = 0.75$, $\text{expand_v} = 1.5$. n_head доходит только до слоёв CausalSelfAttention гибридного варианта H1. Поэтому размерности голов получаются 200/400, а не 128, которые рекомендует раздел аблаций статьи; ровно на $d_k = 128$ попадает конфиг 0.4B ($n_embd = 1536$).

- **Не учтено в столбце состояния:** состояние короткой свёртки на $q/k/v$, $(2 * \text{key_dim} + \text{value_dim}) * \text{conv_size}$ элементов — ещё 57,600 B на каждый слой типа «дельта-правило с гейтом», и это не зависит от n . Формула из раздела «Два числа» этого не включает.
- **Ссылки на источники:**
 - `name = 'GatedDeltaNet_1.3B'`, `n_layer = 16`, `n_head = 16`, `n_embd = 2400`, `gated_delta_per_layer = 1` — NVlabs/GatedDeltaNet lit_gpt/config.py
 - `GatedDeltaNet(hidden_size=config.n_embd)` со значениями по умолчанию слоя $\text{expand_k} = 0.75$, $\text{expand_v} = 1.5$, $\text{num_heads} = 9$ — NVlabs/GatedDeltaNet lit_gpt/model.py, строка 288, и lit_gpt/gated_delta_net.py, строки 34-81
 - $\text{key_dim} = 0.75 * 2400 = 1800 \rightarrow d_k = 200$; $\text{value_dim} = 1.5 * 2400 = 3600 \rightarrow d_v = 400$ — lit_gpt/gated_delta_net.py, строки 72-81
 - 'a head dimension of 128 provides an optimal trade-off between performance and computational efficiency' — arXiv 2412.06464, аблации в приложении

Gated DeltaNet-2

- **Конфиг:** NVlabs/GatedDeltaNet-2, конфиг `gdn2_1.3b`; размерности голов сверены с приложением E.1 arXiv 2605.22791 (опубликованный конфиг обучения (официальный релиз кода; чекпоинта на HF нет)).
- **dtype_bytes = 2** — `pretrain.py: L.Fabric(..., precision='bf16-mixed')`; но см. примечание о типе данных состояния — само рекуррентное состояние статья хранит в `fp32`.
- **Статья:** arXiv 2605.22791 — Gated DeltaNet-2: Decoupling Erase and Write in Linear Attention (Ali Hatamizadeh, Yejin Choi, Jan Kautz; submitted 2026-05-21)
- **Источник конфига:** https://github.com/NVlabs/GatedDeltaNet-2/blob/a5552fe3c67e0ebc7ef1220df68ae8896ec62d56/lit_gpt/config.py
- **Источник реализации:** https://github.com/NVlabs/GatedDeltaNet-2/blob/a5552fe3c67e0ebc7ef1220df68ae8896ec62d56/lit_gpt/gdn2.py
- **Цитирование перепроверено:** 2026-09-05, через arXiv API (`export.arxiv.org/api/query?id_list=2605.22791`), HTML-версию самой статьи и релиз кода на <https://github.com/NVlabs/GatedDeltaNet-2> (HTTP 200). Идентификатор, название и авторы совпадают.
- **Примечание:** Смысл этой строки в том, что она — конечная точка: положив $b_t = \text{beta_t} 1$ и $w_t = \text{beta_t} 1$, получаем в точности KDA, а если вдобавок схлопнуть α_t до скаляра — Gated DeltaNet; так что вся последовательность починок из блока 3 уместается в одно уравнение. Статья записывает обновление ровно в обозначениях модуля: S in $R^{d_k \times d_v}$, стирающий множитель СЛЕВА, $o_t = S_t^T q_t$.
- **Источники расходятся, и вот как это разрешено:** Приложение E.1 статьи указывает `d_model = 2048` и нигде не публикует число слоёв. Релиз кода публикует, и он расходится: `gdn2_1.3b` — это `n_embd = 2304` на 18 слоях. Геометрия голов, задающая размер состояния ($H = 16$, $d_k = d_v = 128$), в обоих одинакова, так что число на слой не затронуто; число по всей модели использует 18 слоёв из релиза кода — единственное опубликованное число слоёв.
- **Тип данных состояния:** Приложение D.3: 'The recurrent state is stored in `fp32` across chunks and during recurrent decoding.' При `fp32` состояние этой строки на слой — 1.00 MiB, а не 512 KiB. В таблице эта строка, как и все остальные, посчитана по 2 байта, чтобы сравнивалась ширина; в реальном развёртывании увидят именно цифру для `fp32`.
- **Не учтено в столбце состояния:** состояние короткой свёртки на $q/k/v$, $(2 * \text{key_dim} + \text{value_dim}) * \text{conv_size}$ элементов — ещё 48 KiB (49,152 B) на каждый слой типа «дельта-правило-2 с гейтом», и это не зависит от n . Формула из раздела «Два числа» этого не включает.
- **Ссылки на источники:**

- ▶ name = 'gdn2_1.3B' (комментарий: Total parameters 1,302,638,112), n_layer = 18, n_head = 18, n_embd = 2304, gdn2_per_layer = 1 (то есть каждый слой — слой GDN-2) — NVlabs/GatedDeltaNet-2 lit_gpt/config.py
- ▶ GatedDeltaNet2(hidden_size=config.n_embd) со значениями по умолчанию слоя expand_v = 1, head_dim = 128, num_heads = 16 -> H = 16, d_k = d_v = 128 — NVlabs/GatedDeltaNet-2 lit_gpt/model.py, строка 212, и lit_gpt/gdn2.py, строки 99-133
- ▶ 'Gated DeltaNet, KDA, and Gated DeltaNet-2 use H = 16 heads with d_k = 128 and d_v = 128, giving a per-layer recurrent state of $H \cdot d_k \cdot d_v = 16 * 128 * 128 = 262,144$ floats per batch element. Since d_model = 2048, this equals 128 d_model.' — arXiv 2605.22791, приложение E.1, ур. (90)
- ▶ Ур. (10), Gated Delta Rule-2 в рамке, и ур. (8), (11), (12) для гейтов — arXiv 2605.22791, раздел 3.1
- ▶ 'The recurrent state is stored in fp32 across chunks and during recurrent decoding.' — arXiv 2605.22791, приложение D.3
- ▶ 1.3B параметров, 100B токенов FineWeb-Edu, длина обучения 4K, окно SWA 2K для гибрида — arXiv 2605.22791, раздел 4 и приложение E.1
- ▶ precision = 'bf16-mixed' — NVlabs/GatedDeltaNet-2 pretrain.py, строка 49

KDA (Kimi Linear)

- **Конфиг:** moonshotai/Kimi-Linear-48B-A3B-Instruct (48B всего, 3B активных, МоЕ) (опубликованный чекпоинт).
- dtype_bytes = 2 — config.json dtype = bfloat16.
- **Статья:** arXiv 2510.26692 — Kimi Linear: An Expressive, Efficient Attention Architecture
- **Источник конфига:** <https://huggingface.co/moonshotai/Kimi-Linear-48B-A3B-Instruct/raw/main/config.json>
- **Примечание:** Читайте соотношение внимательно. Статья говорит 'a uniform 3:1 ratio', а выпущенный конфиг — 20 KDA : 7 MLA на 27 слоях: полное внимание на слоях 4, 8, 12, 16, 20, 24 (это и есть паттерн 3:1 на первых 24 слоях) И на слое 27, последнем. Так что выпущенная модель — 2.86:1, и её число-прирост на 74.1% ниже полного MLA, а не ровно на 75%. Оба округляются до статейного 'up to 75%'.
- **Учёт MLA:** Член, линейный по N, здесь — MLA, который НЕ подчиняется GQA-формуле из раздела «Два числа». MLA кэширует один сжатый латент на токен на слой, шириной kv_lora_rank + qk_rope_head_dim = 512 + 64 = 576, общий для всех 32 голов: 1152 байта на токен на слой MLA. Именно на таком учёте держится заявление о 75%. Эталонная HF-реализация (modeling_kimi.py, KimiMLAAttention.forward) вместо этого материализует в кэш развёрнутые K и V по головам — $32 * (192 + 128) = 10,240$ элементов, 20,480 байт на токен на слой, ~18x больше. Эффективный 'absorbed'-путь, который используется в vLLM, кэширует латент. Цифра 75% определяется соотношением слоёв и одинакова при любом из двух учётов.
- **Не учтено в столбце состояния:** состояние короткой свёртки на q/k/v, $3 * \text{num_heads} * \text{head_dim} * \text{kernel}$ элементов — ещё 96 KiB (98,304 B) на каждый слой типа «KDA (линейное)», и это не зависит от n. Формула из раздела «Два числа» этого не включает.
- **Ссылки на источники:**
 - ▶ num_hidden_layers = 27, linear_attn_config.full_attn_layers = [4, 8, 12, 16, 20, 24, 27], linear_attn_config.kda_layers = 20 элементов, linear_attn_config.num_heads = 32, linear_attn_config.head_dim = 128, linear_attn_config.short_conv_kernel_size = 4, kv_lora_rank = 512, qk_rope_head_dim = 64, qk_nope_head_dim = 128, v_head_dim = 128, num_attention_heads = 32, mla_use_nope = true, dtype = bfloat16 — moonshotai/Kimi-Linear-48B-A3B-Instruct config.json
 - ▶ 'Kimi Linear interleaves KDA with periodic full attention layers in a uniform 3:1 ratio. This hybrid structure reduces memory and KV-cache usage by up to 75% during long-sequence generation' — arXiv 2510.26692, введение
 - ▶ 'reducing KV cache usage by up to 75% and achieving up to 6x decoding throughput for a 1M context' — arXiv 2510.26692, аннотация
 - ▶ 'a uniform 3:1 ratio, i.e., repeating 3 KDA layers to 1 full MLA layer, provided the best quality-throughput trade-off' — arXiv 2510.26692, раздел 4, Hybrid model architecture; абляция в таблице 1 (при 3:1 ppl на обучении 9.23 / на валидации 5.65, против 0:1, 1:1, 7:1, 15:1)

- ▶ 'a fixed-sized state ($d_k \times d_v$ per head, with $d_k = d_v = 128$) regardless of sequence length' — arXiv 2510.26692, раздел 6
- ▶ `kv_a_proj_with_mqa` проецирует в `kv_lora_rank + qk_rope_head_dim`; `past_key_values.update` кэширует развёрнутые `key_states/value_states` — `modeling_kimi.py`, строки 360-414

Qwen3-Next

- **Конфиг:** Qwen/Qwen3-Next-80B-A3B-Instruct (80B всего, 3B активных, MoE) (опубликованный чекпоинт).
- **dtype_bytes** = 2 — `config.json torch_dtype = bfloat16`.
- **Статья:** Qwen3-Next model card; HF transformers implementation
- **Источник конфига:** <https://huggingface.co/Qwen/Qwen3-Next-80B-A3B-Instruct/raw/main/config.json>
- **Источник реализации:** https://github.com/huggingface/transformers/blob/0ed6d51ae8ed3f4fafca67a983b8d75bc76cd51b/src/transformers/models/qwen3_next/modular_qwen3_next.py
- **Примечание:** Линейные слои Qwen3-Next — с точностью до байта эталонный линейный слой лекции: $H = 32$, $d_k = d_v = 128$, 2 байта — 1 MiB на слой. Это не совпадение, которое стоит прятать; это самое чистое место во всей таблице, на которое можно указать. Внимание с группировкой значений: `linear_num_key_heads = 16`, но `linear_num_value_heads = 32`. `q` и `k` умножаются через `repeat_interleave` в 2 раза перед рекуррентностью, так что состояние несёт 32 головы, а не 16. Если посчитать здесь 16, строка уменьшится вдвое.
- **Не учтено в столбце состояния:** состояние короткой свёртки, `conv_dim * linear_conv_kernel_dim` элементов — ещё 64 KiB (65,536 B) на каждый слой типа «Gated DeltaNet (линейное)», и это не зависит от N . Формула из раздела «Два числа» этого не включает.
- **Ссылки на источники:**
 - ▶ `num_hidden_layers = 48`, `full_attention_interval = 4`, `linear_num_key_heads = 16`, `linear_num_value_heads = 32`, `linear_key_head_dim = 128`, `linear_value_head_dim = 128`, `linear_conv_kernel_dim = 4`, `num_attention_heads = 16`, `num_key_value_heads = 2`, `head_dim = 256`, `partial_rotary_factor = 0.25`, `torch_dtype = bfloat16` — Qwen/Qwen3-Next-80B-A3B-Instruct `config.json`
 - ▶ `layer_types = ['linear_attention' if bool((i + 1) % interval_pattern) else 'full_attention' for i in range(num_hidden_layers)]` при `interval_pattern = full_attention_interval = 4`, то есть полное внимание при $i = 3, 7, \dots, 47$: 12 из 48 слоёв, ровно 3:1 — `transformers configuration_qwen3_next.py`, строки 210-217 (коммит 0ed6d51)
 - ▶ `repeat_interleave(num_v_heads // num_k_heads = 2)` для `query/key` перед `chunk_gated_delta_rule`, так что рекуррентное состояние имеет 32 головы по 128×128 — `modular_qwen3_next.py`, строки 597-599 (коммит 0ed6d51)
 - ▶ `conv_dim = key_dim * 2 + value_dim = 128 * 2 + 128 * 32 = 8192` — `modular_qwen3_next.py`, строка 445